# generalManagement MQTT/TLS/JSON

## Interface Design Description

Service ID: *"generalManagement"*

**Abstract**
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the Blacklist system.

# Contents

**1 Documentation edition and implementation scope** **3**

**2 Overview** **3**

**3 Service Operations** **4**
3.1 operation get-log . . . 4
3.2 operation get-config . . . 4

**4 Data Models** **6**
4.1 struct MqttRequestTemplate . . . 6
4.2 struct MqttResponseTemplate . . . 6
4.3 struct ErrorMessage . . . 6
4.4 struct KeyValues . . . 6
4.5 struct LogEntry . . . 6
4.6 struct LogEntryListResponse . . . 7
4.7 struct LogRequest . . . 7
4.8 struct Page . . . 7
4.9 Primitives . . . 7

**5 Go and Rust implementation variants** **9**
5.1 Go implementation . . . 9
5.2 Rust implementation . . . 9

**6 References** **9**

**7 Revision History** **11**
7.1 Amendments . . . 11
7.2 Quality Assurance . . . 11

# 1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

# 2 Overview

This document describes the `generalManagement` service interface of the Blacklist system over MQTT, which provides general management of the Blacklist system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

| Profile type | Type | Version |
|---|---|---|
| Transfer protocol | MQTT | 3.1.1 [1] |
| Data encryption | TLS | 1.3 [2], 1.2 [3] |
| Encoding | JSON | RFC 8259 [4] |
| Compression | N/A | - |
| Semantics | N/A | - |
| Ontology | N/A | - |

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/blacklist/general/management/`. The standalone deployment sets `mqtt.api.enabled=false`, so the MQTT interfaces are not registered there.

# 3 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

| Operation | Request topic |
|---|---|
| `get-log` | `arrowhead/blacklist/general/management/get-log` |
| `get-config` | `arrowhead/blacklist/general/management/get-config` |

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

## 3.1 PUBLISH `arrowhead/blacklist/general/management/get-log`

**Operation:** get-log
**Input:** LogRequest
**Output:** LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

## 3.2 PUBLISH `arrowhead/blacklist/general/management/get-config`

**Operation:** get-config
**Input:** Array<String>
**Output:** KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

# 4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

## 4.1 struct MqttRequestTemplate

The request envelope.

| Object Field | Value Type | Description |
|---|---|---|
| "traceId" | String | Identifier chosen by the requester, returned in the response. |
| "authentication" | String | Identity of the requester (see Section 3). |
| "responseTopic" | String | Topic on which the response is published. |
| "qosRequirement" | Number | MQTT QoS of the response (0, 1 or 2). |
| "params" | Object<String> | Optional parameters (the query parameters of the HTTP binding). |
| "payload" | Object | The operation input. |

## 4.2 struct MqttResponseTemplate

The response envelope.

| Object Field | Value Type | Description |
|---|---|---|
| "status" | Number | Status code. |
| "traceId" | String | The trace identifier of the request. |
| "receiver" | String | Name of the requester. |
| "payload" | Object | The operation output, or an ErrorMessage. |

## 4.3 struct ErrorMessage

| Object Field | Value Type | Description |
|---|---|---|
| "errorMessage" | String | Error text, if the event is an error. |
| "errorCode" | Number | |
| "exceptionType" | ExceptionType | |
| "origin" | String | |

## 4.4 struct KeyValues

Configuration values of the system.

| Object Field | Value Type | Description |
|---|---|---|
| "map" | Object<String> | The configuration keys and values; secrets are not returned. |

## 4.5 struct LogEntry

One log entry.

| Object Field | Value Type | Description |
|---|---|---|
| "logId" | String | Identifier of the log entry. |
| "entryDate" | DateTime | UTC time of the log entry. |
| "logger" | String | Logger (Java class) that wrote the entry; as filter, its name. |
| "severity" | String | Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level. |
| "message" | String | Text of the log entry or status message of the job. |
| "exception" | String | Stack trace or exception text of the log entry, if any. |

## 4.6 struct LogEntryListResponse

List of log entries.

| Object Field | Value Type | Description |
|---|---|---|
| "entries" | Array<LogEntry> | Matching records of the current page. |
| "count" | Number | Total number of matching records (all pages). |

## 4.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

| Object Field | Value Type | Description |
|---|---|---|
| "pagination" | Page | Paging and sorting of the result. |
| "from" | DateTime | Return entries from this UTC time. |
| "to" | DateTime | Return entries until this UTC time. |
| "severity" | String | Minimum log level to return. |
| "logger" | String | Logger (Java class) that wrote the entry; as filter, its name. |

## 4.8 struct Page

Paging and sorting; all fields optional.

| Object Field | Value Type | Description |
|---|---|---|
| "page" | Number | Zero-based page number. |
| "size" | Number | Number of records per page (at most `max.page.size`, default 1000). |
| "direction" | String | Sort direction, `ASC` or `DESC`. |
| "sortField" | String | Field to sort by. |

## 4.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

| Type | Description |
|---|---|
| Array | A JSON Array [4]; Array<A> has elements of type A. |
| DateTime | JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`. |
| ExceptionType | One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String). |
| Number | JSON Number, an integer. |
| Object | JSON Object; Object<A> has string keys and values of type A. |
| String | JSON String. |

# 5 Go and Rust implementation variants

## 5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-Blacklist-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

**Listening port.** Built-in default 127.0.0.1:8464 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8464"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

**Native behavior and outstanding work.** Create/query/revoke, active-time checks, lookup. Golden tests for normalization, historical reactivation and query-mode edge cases. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-Blacklist-go/documentation/Blacklist`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

## 5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-Blacklist-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

**Listening port.** Default 127.0.0.1:8464 (Arrowhead default, role blacklist). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

**Native behavior and outstanding work.** Create/revoke, expiry/check/lookup, filtered queries. Full normalization and historical query parity. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Blacklist`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

# 6 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.3.0, 2026, blacklist-v5.3.0-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (Blacklist),” Eclipse Arrowhead, IDD 5.3.0, 2026, blacklist-v5.3.0-generalManagement-HTTP-IDD.pdf.

# 7 Revision History

## 7.1 Amendments

| No. | Date | Version | Subject of Amendments | Author |
|---|---|---|---|---|
| 1 | 2026-10-22 | 5.2.2 | Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace. | Jerker Delsing |
| 2 | 2026-10-30 | 5.2.2 | Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added. | Claude Opus 5.5 (draft) |
| 3 | 2026-11-09 | 5.3.0 | New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained. | Documentation maintenance |

## 7.2 Quality Assurance

| No. | Date | Version | Approved by |
|---|---|---|---|
| 1 | | 5.3.0 | |